

Запускать так: ./run.sh

Жадный алгоритм и Pilot с bfs обходом

Напишем жадный алгоритм с лекции. Для каждого множества найдём отношение стоимости множества к ещё не покрытым элементам в нём, выберем множество с наименьшим отношением. Различие ответа такого жадного алгоритма и истинного оптимального значения будет не более чем в $H(n) = \frac{1}{n} + \frac{1}{n-1} + \dots + 1$ больше.

Действительно, пусть жадный алгоритм по очереди брал множества $S_1, S_2, \dots, S_s$, а x_i — количество новых элементов покрытых множеством S_i . Пусть $c_i = \frac{\text{cost}(S_i)}{x_i}$ — удельная цена покрытия одного элемента выбором S_i . Полная стоимость жадного решения будет равна $\sum_{i=1}^s \text{cost}(S_i) = \sum_{i=1}^s c_i x_i$. Присвоим всем элементам цену, элементам впервые добавленным множеством S_i присвоим цену c_i , тогда сумма всех стоимостей есть $\sum_{i=1}^s c_i x_i = \text{cost}(\{S_1, \dots, S_s\})$.

Рассмотрим множество A из оптимального набора. Пусть $A = \{a_1, a_2, \dots, a_k\}$, упорядочим их в порядке добавления жадным алгоритмом. Рассмотрим a_i , на момент его добавления жадным алгоритмом ещё не были покрыты $a_i, a_{i+1}, \dots, a_k$, то есть удельная цена A была $\frac{\text{cost}(A)}{k-i+1}$, при этом была выбрана наименьшая возможная, то есть $\text{cost}(a_i) \leq \frac{\text{cost}(A)}{k-i+1}$. Суммируя по всем элементам из A получается $\sum_{u \in A} \text{cost}(u) \leq \text{cost}(A) \left(1 + \frac{1}{2} + \dots + \frac{1}{|A|}\right) \leq \text{cost}(A) H(n)$. Заметим, что неравенства достаточно грубые и предполагают, что множества дизъюнкты, а оценка на стоимость элемента предполагает, что элементов других множеств нет.

Реализуем эту эвристику без ничего дополнительного. В лобовой реализации она даёт следующие метрики

102100, 30, 170, 36, 76, 192.

Что даёт 15 баллов 3, 0, 3, 3, 3, 3.

Несложное её улучшение заключается в том, что жадный алгоритм не гарантирует, что первые множества после полностью не перекрылись более поздними, так что на всякий случай попытаемся удалить каждое из множеств ответа, если вдруг все его элементы лежат в каких-то других множествах ответа. Это действительно немного улучшило метрики, они составили

102100, 30, 156, 32, 69, 181.

Что также даёт 3, 0, 3, 3, 3, 3, хоть и улучшило метрики.

Далее был написан Pilot method, мы попытались пройти все возможные способы взять или не взять множества. Так как есть гарантия на небольшое отклонение от наилучшего возможного ответа, то ветку можно закрывать, если значение жадного алгоритма делённого на H уступает наилучшему значению. Однако, как было отмечено оценка довольно грубая, так что при нахождении числа, которое может быть добавлено в подветку считать будем не само значение эвристики делённое на H , а домноженное на $\frac{1}{\alpha H^\beta}$, где α, β — настраиваемые параметры.

Уменьшение значения α позволяет не заходить в часть веток, если мы немного не добиваем до гарантии, что в ветке ответ будет не лучше, а также не верим, что оценка может оказаться точной, этот параметр скорее отвечает за то, сколько раз каждый элемент в среднем покрыт множествами наилучшего решения. Параметр β скорее отвечает за то, насколько грубая является оценка $\text{cost}(a_i) \leq \frac{\text{cost}(A)}{k-i+1}$, которая предполагает отсутствие других множеств

оптимального семейства, то есть вера в то, что H должно вычислять от значения в предположении, что $cost(a_i) \leq \frac{cost(A)}{k-i+1+C}$, то есть от самого H оставлять лишь легковесный хвост этой суммы. При смещении мы хотели бы заменить значение с $\ln(n)$ на $\ln(n+C) - \ln(C) = \ln(1 + \frac{n}{C})$, где C — количество элементов не из множества. Предположим, что n — какая-то функция от C и количество непокрытых множеств k , то есть $n = \frac{C}{k}$, а значит мы хотим заменить $\ln(\frac{C}{k}) = \ln(\frac{k+1}{k} \cdot \frac{C}{k+1}) = \ln(1 + \frac{1}{k}) + \ln(\frac{C}{k+1})$ на $\ln(1 + \frac{1}{k})$, где $\ln(\frac{C}{k+1})$ вероятно сильно больше $\ln(1 + \frac{1}{k})$, причём не в константу раз, следующая по скорости уменьшения функция это возведение в степень меньше 1, её и попробуем подобрать.

Без настройки, то есть при $\alpha, \beta = 1$ и 10 секундах на Pilot метрики составляют

96600, 25, 156, 32, 69, 181

Что даёт 18 баллов взяв все маленькие пороги 3, 3, 3, 3, 3.

Попытаемся настроить параметры. Запустимся при $\alpha = 0.2, 0.4, 0.6, 0.8$ и взглянем на ответы. При $\alpha = 0.2$ мы потеряли полезные ветки на втором тесте, но при маленьких α успели

α	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
0.2	96100	27	154	32	69	181
0.4	95800	25	156	32	69	181
0.6	96600	25	156	32	69	181
0.8	96600	25	156	32	69	181

больше получить на первом и третьем. α отвечало за покрытие одного элемента много раз, выведем количество таких покрытий в жадном решении: 1, 3.104, 2.150, 2.944, 3.114, 2.613, то есть маленькие значения α давали больше профита на маленьких значениях среднего перекрытия. Это несколько контринтуитивно, однако попробуем проставить α равным среднему перекрытию, делённому на некоторую константу: после экспериментов наилучшие метрики оказались на константе 5, они составили

94800, 25, 156, 32, 69, 181.

Что всё ещё 18 баллов, взяв все нижние пороги, но результат первого теста существенно улучшился.

Теперь подберём β , тоже запусимся с разными, выберем наилучшее, после чего подвтрим оптимизация α как и ранее.

β	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
0.2	102100	27	156	32	69	181
0.4	101700	27	156	32	69	181
0.6	100800	27	154	32	69	181
0.7	100000	25	153	32	69	181
0.8	98400	25	156	32	69	181
1.2	95800	25	156	32	69	181

Первый тест очень сильно зависит от α , второй и третий лучше всего показали себя при 0.7, поставим β такой и перенастроим делить в α . После настройки оказалось, что с третьим тестом нам повезло, и такое получилось при одной конкретном α , а на первый тест действительно β далёкие от 1 получаются хуже, так что вернём как было.

Запустим наш алгоритм на 60 секунд вместо 10, чтобы понять есть ли смысл отсекалть какие-то ещё ветки, или чуть больше вершин дерева нам пока не помогут. Метрики на них составили:

94800, 25, 153, 32, 69, 181

В целом, от отсечения пары ветвей, которые мы при шестикратном увеличении времени скорее всего успели пройти не поможет, так что будем считать, что в этом случае мы выжали всё что можно было.

Второй способ обхода

В прошлый раз, мы решали идти или не идти по вершине в наперёд заданном порядке определённом в начале. Попробуем реализовать другой обход и настроить его.

Раньше мы вначале посортировали множества по важности и раскрывали их, теперь давайте раскрывать множества не в порядке их удельной стоимости в начале, а множество с самой большой удельной стоимостью на момент раскрытия. Это сделает процедуру раскрытия дольше, так как придётся их вычислить, но возможно выбору будут более важными.

До настройки параметров в таком обходе метрики составили

97000, 29, 156, 32, 69, 181.

Что также, как и раньше, берёт все тесты на нижние пороги.

Попробуем настроить параметры: В целом та же самая картина, аналогично тому, как и

α	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
0.2	95600	29	153	32	69	181
0.4	95800	29	153	32	69	181
0.6	96600	29	156	32	69	181
0.8	96600	29	156	32	69	181

раньше настроим параметры, оптимальной оказалась та же самая настройка, на ней метрики составили

94800, 29, 153, 32, 69, 181.

То есть в целом за 10 секунд был получен хороший результат на третьем тесте и провалился результат на втором, из-за того, что мы прошли меньшую часть дерева.

Также один Pilot может дать ответ, чтобы помочь второму делать более грамотные отсечения, из-за хорошего стартового решения, попытаемся запустить их оба, на 10 секунд каждый.

Порядок имеет значение, так что попробуем оба порядка. Если сначала запустить первый, а потом второй, то метрики составили

94800, 25, 153, 32, 69, 181.

Если сначала второй, а после первый, то метрики составили

94800, 25, 153, 32, 69, 181.

В целом результаты такие же, так что пожалуй, из этого метода мы выжали максимум, а улучшение не было, был просто взят лучший из ответов.

На данный момент при запуске на все 60 секунд максимального времени метрики составляют

94800, 25, 153, 32, 69, 181.

То есть вообще не изменяются от увеличения времени и дают 18 баллов 3, 3, 3, 3, 3, 3.

Локальный поиск и отжиг

На многих тестах мы были близки к большому порогу, а также при наших прошлых способах обхода дерева последние решения почти всегда принимал жадник, а мы успевали назначить лишь верхние значения, попытаемся написать локальные улучшения и дожать имеющиеся ответы.

Первое локальное изменение будет следующим: попытаемся добавить множество и удалить какие-то множества, элементы в которых и так перекрыты кем-то другим. Если решение стало лучше, то применим его, иначе отклоним и вернём как было. Эта операция была успешно применена ко второму, пятому и шестому тестам, метрики составили

94800, 23, 153, 32, 68, 175.

Что даёт 20 баллов 3, 5, 3, 3, 3, 3.

Далее была добавлено локальное изменение на небольшую перерешку, а именно, удалим насильно какое-то множество и попытаемся перекроить иначе элементы этого множества.

К сожалению на решении получившихся на прошлом шаге такое применить нельзя ни разу. Метрики не изменились.

Поскольку пока алгоритм сходится, увеличим окрестность, чтобы он не сходился за имеющееся время.

Разрешив добавлять сразу два элемента метрики улучшились на первом и шестом тестах, при этом на четвёртом, пятом, шестом алгоритм изначально не уложился во время, и пришлось устанавливать ограничение по времени работы. Метрики составили

94600, 23, 153, 32, 68, 170.

Всё те же 20 баллов.

Повышать число добавляемых элементов неоправданно дорого, при этом перебирая не влоб теряется улучшение на шестом тесте. Чтобы обойтись только OneAdd и OneRem напишем отжиг.

Причём OneAdd работает за линию добавляя элемент и подчищая лишнее, в то время как OneRem более дорогостоящая операция, и скорее нужна для разнообразия возможных путей.

Отключим Pilot и начнём настраивать отжиг, сперва выберем необходимую вероятность OneAdd, поэкспериментируем с различными значениями.

<i>add_prob</i>	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
0.5	94400	23	147	31	64	174
0.65	94400	23	147	31	65	174
0.8	94400	23	146	32	66	175
0.95	94400	23	146	31	68	175
0.995	94400	23	149	31	66	170

То есть в целом от большого *add_prob* решения получаются похуже из-за меньшей вариативности, а когда получаются хорошие метрики, они неустойчивы и пропадут при перезапуске, при этом скорость работы увеличивается. Выберем *add_prob* = 0.7, где потери ещё были не велики, но алгоритм уже начинал работать быстрее.

Теперь начнём подбирать температуру. Температура должна быть достаточной на больших, средних и маленьких вероятностях принять ухудшение. Для этого попробуем подобрать параметры так, чтобы было много итераций с вероятностью принять ошибку < 0.01 , > 0.9 , и

из интервала (0.2, 0.8). Для этого надо понимать среднюю δ , поскольку у нас есть много экспериментов и много значений в каких-то локальных минимумах, мы видим, что при ответе порядка 100000 дельта бывает (100, 200), при значениях (20, 50) изменение около 1, при значении 150, 175 это скорее от 2 до 5. Примерно такой природой обладает функция $\sqrt{\frac{x}{10}}$, так что для приближённого ответа найденного эвристикой будем считать $\sqrt{\frac{ans}{10}}$ средним значением дельты.

Теперь замеряем время работы OneAdd и OneRem, вычислим среднее время на запрос к ним. Зная сколько в среднем занимает шаг можно найти количество шагов, которое мы хотели бы сделать в каждом промежутке. Изменение найдём по центральному промежутку: за средний промежуток значение $\frac{\delta}{t}$ должно измениться от $\ln(0.2)$ до $\ln(0.8)$, то есть $q^{med_iter} = \frac{\ln(0.8)}{\ln(0.2)} \Leftrightarrow mex_iter \log(q) = \log(\ln(0.8)) - \log(\ln(0.2))$. Стартовая температура и отсечка по температуре соответственно определяются тем, сколько итераций мы хотим сделать.

Однако компилятор не сумел адекватно замерить время работы OneAdd и OneRem, так что настроим придерживаясь этих формул, но на глаз. Возьмём какой-то temp_change, а начальную и конечную температуру подберём как среднюю дельту умноженную или деленную на какие-то константы, их и подберём.

В ходе настройки мы изменили философию, подберём коэффициенты так, чтобы температура в начале, делённая на среднюю дельту была порядка 100, а в конце порядка 0.01 и лучше уделить всё время среднему классу, а не каждому классу свою часть, как мы пытались сделать.

Действительно, если выделить часть времени на вероятности > 0.9 или на < 0.01 , то метрики не улучшаются, а из-за меньшего числа шагов даже падают. Под такие требования ближе всего подошли начальная температура $15\Delta\sqrt[3]{nm}$, а конечная $\frac{\Delta}{30\sqrt[4]{nm}}$. Также время работы на последних трёх тестах слишком велико, чуть ускорим сходимость, чтобы укладываться во время на $n > 2000$, а также, от очень малого n отжиг работает слишком быстро и можно успеть больше итераций, искусственно добавим при $n < 50$.

Метрики составили

94400, 23, 147, 31, 66, 175.

Что даёт 26 баллов 5, 5, 5, 5, 3, 3.

Проведём эксперимент с начальной точкой. Сейчас она получается жадным алгоритмом. Попробуем брать случайное покрытие множествами. Случайную перестановку будем получать так: у нас уже есть функция, которая удаляет множество, если универсум покрыт и без него. Положим все множества в покрытие, сейчас любое валидное не улучшаемое тривиальным удалением решение можно получить удалив лишнее. Так и будем действовать: сгенерируем случайную перестановку и попытаемся удалить элементы в порядке этой перестановки.

Посмотрим на метрики на одном перезапуске: При старте с жадного результаты чуть

способ генерации	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
случайный	94600	25	149	32	68	175
случайный	94600	23	149	32	65	175
случайный	94400	27	149	32	68	175
случайный	94400	25	149	32	68	175
жадный	94400	25	147	32	68	172
жадный	94600	23	149	32	66	175
жадный	94600	23	147	32	67	175
жадный	94800	23	149	31	68	175

лучше, оставим старт с жадного решения.

LNS

Мы получаем достаточно много решений, близких к верхнему порогу, однако не добираем сам порог, попытаемся перетекать из одного решения в другое, пытаясь найти значение лучше в их комбинации.

Более конкретно: возьмём два решения с хорошей метрикой, найдём по каким множествам они различаются. Начнём будем преобразовать первое решение до второго, для этого будем пытаться удалить множества первого решения, если просто так удаление сделать нельзя, то попытаемся разрешить это удаление с помощью добавления множеств из второго решения.

Найдём способ получения достаточно точных ответов к которым можно пытаться применить процедуру. Попытаемся применить для этого отжиг, но ускорим его, чтобы получать не такое хорошее решение, но получать его значительно быстрее.

Настроив температуру найдём хэмингово расстояние между имеющимся ответом и ответом после перезапуска отжига. В целом вышло порядка 10-50, решения отжига и будем использовать.

Сперва отжиг был отключен и запущен такой алгоритм на 20 секунд. В целом дал метрики не хуже сильно сэкономив время. Метрики составили

94440, 23, 147, 31, 68, 175.

Что дало 26 баллов как 5, 5, 5, 5, 3, 3. Из минусов улучшение было получено именно за счёт перетекания решений не более чем 2 раза на каждом тесте, то есть скорее сыграло большое количество легковесных отжигов, в то время как ранее было мало тяжёлых отжигов.

При перезапуске на минуту метрики не изменились.

На последок было попробовано запустить на сэкономленное время обход дерева с помощью двух Pilot из первого решения, с предположением, что отсечений будет больше из-за уменьшившегося ответа.

Метрики составили

94440, 23, 147, 31, 68, 174.

26 баллов как 5, 5, 5, 5, 3, 3. Видимо методами, основанными на отжиге взять два последних теста не выйдет.

Итог

За большое время наилучшими оказались метрики тяжёлого отжита, они составили

94400, 23, 147, 31, 66, 175,

что даёт 26 баллов 5, 5, 5, 5, 3, 3, есть альтернативное решение через LNS и легковесный отжиг, работающее быстрее, но чаще получающее 32 вместо 31 на четвёртом тесте.